

Kubernetes Incident Recovery Runbook

Scenario

This runbook provides operational steps for diagnosing and recovering Kubernetes application failures.

Pod Crash Troubleshooting

Check Pod Status

To view all pods in a namespace:

```
kubectl get pods -n production
```

Describe a Failed Pod

To inspect events and error messages:

```
kubectl describe pod <pod-name> -n production
```

View Pod Logs

To check application logs:

```
kubectl logs <pod-name> -n production
```

For multi-container pods:

```
kubectl logs <pod-name> -c <container-name> -n production
```

Restart a Deployment

To restart an application deployment:

```
kubectl rollout restart deployment <deployment-name> -n production
```

Verify rollout status:

```
kubectl rollout status deployment <deployment-name> -n production
```

Scale Application

Increase replicas:

```
kubectl scale deployment <deployment-name> --replicas=5 -n production
```

Check replica status:

```
kubectl get deployment <deployment-name> -n production
```

Node Health Check

Check cluster nodes:

```
kubectl get nodes
```

Describe node details:

```
kubectl describe node <node-name>
```

Recover a Failed Node

Drain workloads from a node:

```
kubectl drain <node-name> --ignore-daemonsets
```

Mark node schedulable again:

```
kubectl uncordon <node-name>
```

Check Kubernetes Events

View recent cluster events:

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

Disk Pressure Investigation

Identify nodes under disk pressure:

```
kubectl describe nodes | grep DiskPressure
```

Clean unused Docker images:

```
docker system prune -a
```

Emergency Rollback

Rollback to the previous deployment revision:

```
kubectl rollout undo deployment <deployment-name> -n production
```

Verify rollback:

```
kubectl rollout history deployment <deployment-name> -n production
```